Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

JAVA ASSIGNMENT

NAME:Kavya Gada

SAP ID:60017230102

ROLL NO.:A018

BRANCH:AIML

BATCH:A1

DATE OF SUBMISSION:27/04/24

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Kavya Gada
A018
60017230102

Page
Date: / /

Exp - no. 13

Aim: To implement Multithreading :-

Description of Experiment

Thread class, Runnable interface :-

Threads: A thread is smallest executable piece of code. It allows program to run multiple tasks concurrently. The thread class in Java provides basic functionality for working with Threads.

Runnable Interface: This interface defines a single abstract method named "run()". Any that implements Runnable can be executed by a thread. You define the code you want be run within the "run()" method

Creating a thread :-

- Implement Runnable or Extend thread
- Override run() method
- Create a thread object

Runnable :-

```
My class run = new MyClass();
thread t = new Thread(run);
```

SUN-DEEP

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Page
Date: / /

Thread class :-
My Thread t = new Thread();

Thread Lifecycle :-

New : A thread object has been created but not started
Runnable : The thread is ready to be run when schedule select it.
Running : The thread currently executing run method
Waiting / Time waiting / Blocked : The thread is waiting for an event to occur or is sleeping for a specified duration.
Terminated : The thread has finished executing its run method

Methods of Thread Class :-

- start() : starts the thread execution
- run() : Defines the code to be executed by the Thread.
- sleep(long milliseconds) : suspends the thread for specified time.
- join() : waits for the thread to finish its execution.
- getName() :-

Thread Sychronization : Achieving Mutual Exclusion

SUN-DEEP

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Page
Date: / /

Synchronizat^n ensures that only one thread can acess a shared resource at a time. This prevents race condition where multiple threads could modify the resource in an unexpected way.

1. Sychronized Methods.
- Declared a method as sychronized to guarantee that only one thread can execute at a time.
- Other threads attempting to access the sychronized method will be blocked until the current threads exits the method.

2. Sychronized Blocks:-
- Use the synchronised keyword with an an object to create a critical section within a block of code.
- Only one thread can execute with the synchronized block at a time while holding the lock on the specific object.

3) Static Synchronization
- Declare a method as synchronized static to sychronize access to the entire class (not just instances).
- Only one thread can execute within a static method at a time regardless of how many instances of the class exist.

SUN-DEEP

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

[Type here]

Page
Date: / /

When to use Each Technique:-

- Synchronized methods: But too methods that modify instance variable of class
- Synchronized block: More flexible, allowing you to sychronize only a portion of code that needs to be thread-safe.
- Static Synchronization:- Use with caution; it can severely limits concurrency is overused. Consider using instance. Synchronization most of the time.

Conclusion:- Understanding Threads, runnable interface, thread lifecycle, methods and synchronization in Java.

SUN-DEEP

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

[Type here]

Page
Date: / /

Code :-

```
class One extends Thread {
    public void run() {
        for (int i=0; i<=4; i++) {
            System.Out.print ("*");
            try { Thread.sleep (2000); }
            catch (InterruptedException e) { } }
}

class Two extends Thread
{   public void run() {
        for (int i=0; i<=4; i++) {
            System.out.print ("/");
            try { Thread.sleep (2000); }
            Catch (Exception e) } }
}

class Main
{   public static static void main (String [] args) {
        One o = new One();
        Two t = new Two();
        Thread t1 = new Thread (o);
        Thread t2 = new Thread (t);
        t1.start();
        t2.start();
} }
```

Output :-

P.T.O

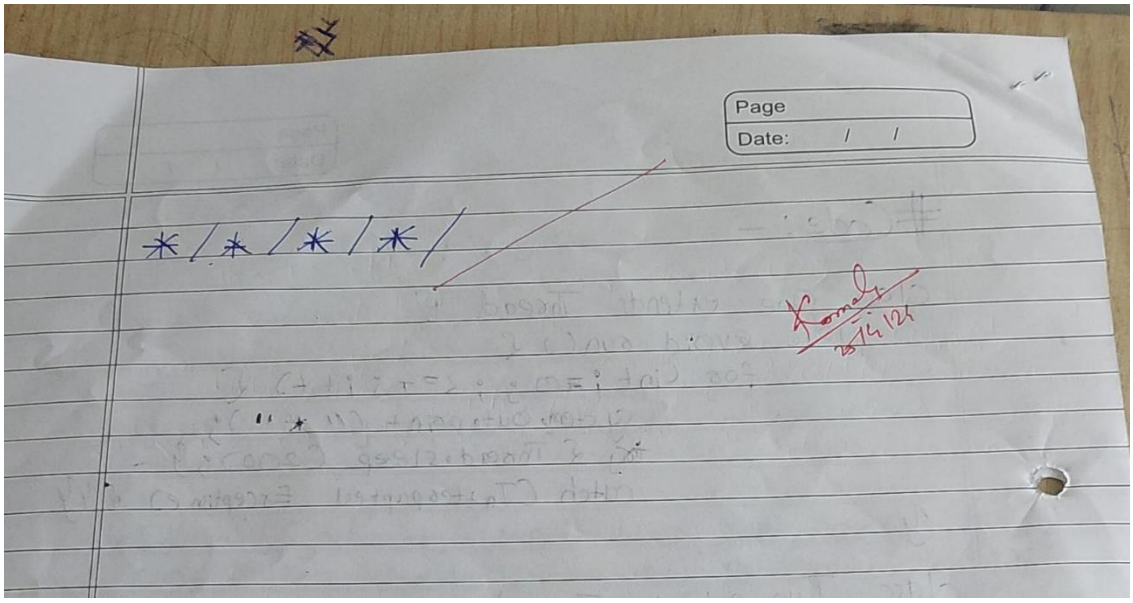
SUN-DEEP

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

[Type here]



[Type here]

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Experiment No: 13

Problem Statement 1: Write a multithreaded program a java program to print Table of Five, Seven and Thirteen using Multithreading (Use Thread class for the implementation).

Code:

```
import java.util.*;
class TableOfFive extends Thread
{
public void run()
{
for(int i=1;i<=10;i++)
{
System.out.println("5 x "+i+" = "+(i*5));
}}}
class TableOfSeven extends Thread
{
public void run()
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
{
for(int i=1;i<=10;i++)
{
System.out.println("7 x "+i+" = "+(i*7));
}}}
class TableOfThirteen extends Thread
{
public void run()
{
for(int i=1;i<=10;i++)
{
System.out.println("13 x "+i+" = "+(i*13));
}
}
}
class Threads
{
public static void main(String args[])
{
TableOfFive five=new TableOfFive();
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
five.start();
TableOfSeven seven=new TableOfSeven();
seven.start();
TableOfThirteen thirteen=new TableOfThirteen();
thirteen.start();
}
}
```

OUTPUT:

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\java program

C:\java program>javac Threads.java

C:\java program>java Threads
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
5 x 1 = 5
5 x 2 = 10
13 x 1 = 13
5 x 3 = 15
13 x 2 = 26
5 x 4 = 20
13 x 3 = 39
5 x 5 = 25
13 x 4 = 52
5 x 6 = 30
13 x 5 = 65
5 x 7 = 35
13 x 6 = 78
5 x 8 = 40
13 x 7 = 91
13 x 8 = 104
5 x 9 = 45
13 x 9 = 117
5 x 10 = 50
13 x 10 = 130

C:\java program>_
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Problem Statement 2: Write a multithreaded program to display /*/*/*/*/*/*/* using 2 child threads.

Code:

```
class Slash extends Thread {
public void run() {
for(int i=0;i<10;i++)
{
try {
System.out.print("/");
Thread.sleep(199);
}
catch(InterruptedException e)
{
System.out.println(e);
}}}}
class Star extends Thread
{
public void run() {
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
for(int i=0;i<10;i++)
{
try {
System.out.print("*");
Thread.sleep(200);
}
catch(InterruptedException ie)
{
System.out.println(ie);
}}}}
class ThreadsB {
public static void main(String args[]) {
Slash slash=new Slash();
Star star=new Star();
slash.start();
star.start();
}}
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

OUTPUT:

```
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\java program

C:\java program>javac ThreadsB.java

C:\java program>java ThreadsB
/**/*/*/*/*/*/*/*/
C:\java program>
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Problem Statement 3: Write a program to demonstrate thread methods: wait notify suspend resume join setpriority getpriority setname getname.

Code:

```
class A extends Thread
{
public void run()
{
String name=Thread.currentThread().getName();
System.out.println("\n-_- -_- -_- -_-");
for(int i=0;i<5;i++)
{
System.out.println("\thello "+ name);
}}}
class ThreadsC
{
public static void main(String args[])
{
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
A a1=new A();
A a2=new A();
System.out.println("Initial thread
name:"+a1.getName());
System.out.println("Initial thread
name:"+a2.getName());
a1.setName("A1");
a2.setName("A2");
System.out.println("Changed thread
name:"+a1.getName());
System.out.println("Changed thread
name:"+a2.getName());
System.out.println("\nInitial priority of
A1:"+a1.getPriority());
System.out.println("Initial priority of
A2:"+a2.getPriority());
a1.setPriority(10);
a2.setPriority(7);
System.out.println("Changed priority of
A1:"+a1.getPriority());
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
System.out.println("Changed priority of A2:"+a2.getPriority());
a1.start();
a2.start();
}
}
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Output:

```
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\java program

C:\java program>javac ThreadsC.java

C:\java program>java ThreadsC
Initial thread name:Thread-0
Initial thread name:Thread-1
Changed thread name:A1
Changed thread name:A2

Initial priority of A1:5
Initial priority of A2:5
Changed priority of A1:10
Changed priority of A2:7

-_- -_- -_- -_-

-_- -_- -_- -_-
        hello A1
        hello A1
        hello A1
        hello A1
        hello A1
        hello A2
        hello A2
        hello A2
        hello A2
        hello A2

C:\java program>_
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Problem Statement 4: Write a multithreaded program that generates the Fibonacci sequence. This program should work as follows: create a class Input that reads the number of Fibonacci numbers that the program is to generate. The class will then create a separate thread that will generate the Fibonacci numbers, placing the sequence in an array. When the thread finishes execution, the parent thread (Input class) will output the sequence generated by the child thread. Because the parent thread cannot begin outputting the Fibonacci sequence until the child thread finishes,the parent thread will have to wait for the child thread to finish.

Code:

```
import java.util.Scanner;
class Input extends Thread
{
int arr[],n;
Scanner ob=new Scanner(System.in);
Fibonacci fib;
Input()
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
{
System.out.println("Enter number of elements to be
displayed:");
n=ob.nextInt();
arr=new int[n];
fib=new Fibonacci(arr,n);
}
public void run()
{
try
{
fib.run();
fib.join();
}
catch(InterruptedException e)
{
System.out.println(e);
}
System.out.println("Fibonacci Sequence:");
for (int i = 0; i <= arr.length - 1; i++)
{
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
System.out.print(arr[i] + " ");
}
}
}
class Fibonacci extends Thread
{
int arr[], n;
Fibonacci(int arr[], int n)
{
this.arr=arr;
this.n=n;
}
public void run()
{
int a=0, b=1,c;
arr[0]=a;
arr[1]=b;
for(int i=2;i<n;i++)
{
c=a+b;
arr[i]=c;
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
a=b;
b=c;
}
}
}
class ThreadD
{
public static void main(String args[])
{
Input in=new Input();
in.start();
}
}
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

OUTPUT:

```
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\java program

C:\java program>javac ThreadD.java

C:\java program>java ThreadD
Enter number of elements to be displayed:
8
Fibonacci Sequence:
0 1 1 2 3 5 8 13
C:\java program>_
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Problem Statement 5: WAP to prevent concurrent booking of a ticket using the concept of thread synchronization.

Code:

```
import java.util.Scanner;
class Booking
{
int tickets=20, n;
synchronized public void getTickets()
{
Scanner ob=new Scanner (System.in);
System.out.println("\nAVAILABLE TICKETS: "+tickets);
System.out.println("Enter number of tickets
required:" );
n=ob.nextInt();
checkTickets(n);
}
public void checkTickets(int n)
{
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
if(tickets==0)
System.out.println("TICKETS SOLD OUT!");
else if (n> tickets)
System.out.println("Tickets Unavailable");
else
{
System.out.println(n+ " tickets booked");
tickets-=n;
}
}
}
class A extends Thread
{
Booking b;
A(Booking b)
{
this.b=b;
}
public void run()
{
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

```
b.getTickets();
}
}
class Last
{
public static void main(String args[])
{
Booking b= new Booking();
A a1=new A(b);
A a2=new A(b);
A a3=new A(b);
A a4=new A(b);
A a5=new A(b);
a1.start();
a2.start();
a3.start();
a4.start();
a5.start();
}
}
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Output:

```
Microsoft Windows [Version 10.0.19045.4291]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\java program

C:\java program>javac Last.java

C:\java program>java Last

AVAILABLE TICKETS: 20
Enter number of tickets required:
4
4 tickets booked

AVAILABLE TICKETS: 16
Enter number of tickets required:
7
7 tickets booked

AVAILABLE TICKETS: 9
Enter number of tickets required:
1
1 tickets booked

AVAILABLE TICKETS: 8
Enter number of tickets required:
9
Tickets Unavailable

AVAILABLE TICKETS: 8
Enter number of tickets required:
8
8 tickets booked

C:\java program>
```


NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018

Conclusion:

- **Thus we have studied and learnt about multithreading and synchronization.**
- **We also learnt about the different methods used in threads**

NAME:Kavya Gada

SAP ID: 60017230102

ROLL NO.:A018